Rule-based rewriting of REGEX

Into LIKE

Anjesh Narwal, 24421

Amandeep Nokhwal, 24144

December 2024

1. Introduction

As the amount of data stored in databases is getting increased day by day, query processing becomes a crucial component in database systems. Among the various operations performed on databases, pattern matching is one of the frequently used features, particularly in applications like data validation, searching, and text analysis. SQL provides two primary tools for pattern matching: the LIKE operator and regular expressions (REGEXP or REGEXP_LIKE in Oracle DB). While regular expressions offer powerful and flexible pattern matching capabilities, they often come with significant computational overhead. This project focuses on converting regex-based queries into simpler LIKE queries wherever possible, addressing the need to optimize query processing in relational database systems. Also, we have used Oracle DB for testing and comparing the time taken by original complex regex query and our transformed query as Oracle DB have both the operators LIKE as well as REGEXP_LIKE. The REGEXP_LIKE operator in Oracle DB provides rich pattern extraction features such as character classes, occurrence indicators, meta-characters, etc.

1.1 The Problem with REGEX Queries

Regular expressions provide a robust mechanism to match complex patterns in string data, supporting features like character classes, repetitions, alternations, and more. For example, a regex pattern for matching email addresses,

^[\w\-\.\+]+\@[a-zA-Z0-9\.\-]+\.[a-zA-Z0-9]{2,4}$

, can accurately validate emails. However, this power comes at a cost:

1. **Computational Expense**: Regex engines typically rely on backtracking algorithms or finite state automata, which may result in non-linear execution times for complex patterns. This can severely impact performance, especially for large datasets. Complex patterns with alternations, nested groups, or backreferences can significantly slow down execution due to extensive backtracking.
2. **Indexing Challenges**: Queries using regular expressions often bypass database indexes due to their dynamic nature, requiring full table scans. This further reduces the query performance.

The LIKE operator is less expressive than regular expressions. LIKE operator only offers wildcard matching (The percent sign % represents zero, one, or multiple characters and the underscore sign _ represents one, single character). It provides significant performance advantages for simpler pattern matching tasks:

1. **Index Utilization**: Queries using LIKE with leading literals (e.g., LIKE 'abc%') can leverage database indexes, enabling faster lookups and reducing the need for full table scans.
2. **Lower Computational Overhead**: The LIKE operator is implemented with simple substring matching algorithms, which are computationally lightweight compared to regex engines.

1.2 Benefits of transforming the REGEX query into simpler LIKE statements

The cost of ineffective queries increases as the amount of data held in databases increases. A native regex search often involves extensive processing and can significantly slow down query execution times. By strategically transforming regex patterns into LIKE queries, this project aims to strike a balance between functionality and performance, ensuring that query results are returned with minimal delay while preserving accuracy. As the expressive power for regex is more than the expressive power for LIKE operator entire regex query can't be converted to equivalent LIKE statement i.e. any regex which involved repetition matching cannot be expressed by LIKE due to its lack of structural repetition handling.

2. Our Approach

If we try to split the query into two or more different queries i.e. few parts handlings what could be handled only by LIKE operators and other parts handling complex pattern matching i.e. repetition of a group via lesser complex regex than the input regex, then we won't be able to prove that after combining the results from these smaller conditions we will get the same result as original regex. For e.g. our input query was

SELECT string_column FROM TABLE WHERE REGEXP_LIKE(string_column, `^abc*$`)

The string accepted by this query will be from this set {ab, abc, abcc, abccc, abcccc, ...etc.}. If we try to transform it into simpler query, the subpart that can be identified by LIKE is that string should start from ab, that is string_coulmn LIKE `ab%` and we cannot use LIKE to identify string ending with zero or more occurrence of c, so other subpart can be REGEXP_LIKE(string_column, `c*$`) and to combine these two conditions we can use AND operator.

SELECT string_column FROM TABLE WHERE string_column LIKE `ab%` AND REGEXP_LIKE(string_column,`c*$`)

But these two queries are not equivalent, the second query will accept string which are starting from `ab` and ending with zero or more occurrence of character `c` but it can also have other characters in between which were not included in the original query, i.e. `abdfcc`. We have to include the conditions checked by LIKE operator in REGEXP_LIKE operator to get equivalent result. Hence the transformation which will work-

SELECT string_column FROM TABLE WHERE string_column LIKE `ab%` AND REGEXP_LIKE(string_column,` abc*$`)

If the regex query can entirely be converted into LIKE statements i.e. there is no troubling operator, then this algorithm will convert the input regex to LIKE statements separated by OR.

Input query-

SELECT string_column FROM TABLE WHERE REGEXP_LIKE (string_column, `^ab[de]$)

Transformed Query-

SELECT string_column FROM TABLE WHERE (string_column LIKE `abd` OR string_column LIKE `abe`) .

The natural question how will this transformation help in reducing the execution time of the query. When the original REGEXP_LIKE query is executed, the engine parses the pattern into an internal representation, typically an Abstract Syntax Tree (AST) or a finite automaton. Each candidate row in the column being queried is processed against the regex. The regex engine evaluates the string in the column against the compiled regex pattern. But for the second query for the rows where string_column is not starting with `ab` due to the AND operator, short circuiting will be performed by the compiler and the regex checking won't be performed, hence it will be fast.

2.1 Unconvertable patterns

OBSERVATION: In the regex if we have two repeating groups and in between there is a fixed pattern to match, i.e. (abc)*d(gh)* . We cannot represent the fixed patten (`d` here) with LIKE statement. This is because d should be preceded by zero or more occurrence of `abc` and followed by zero or more occurrence of `gh`. Hence in LIKE we can't use `_` before or after d as we don't know how many of `abc` and `gh` will be there and similarly we can't use `%` before or after as it will result in unnecessary strings getting accepted.

2.2 Query transformation workflow

Hence, we can generalize that between any two repeating group patterns (i.e. * for 0 or more occurrence, + for one or more occurrence, {n,m} for occurrence between n and m, as these cannot be handled by LIKE statement, let's call them troubling operators) any pattern or operator or set of operators can also not be handled by only using the LIKE statement. So, we will identify the first troubling operator from the right as well as the left of the regex pattern. Part of regex before first troubling operator can be handled by combination of LIKE statements (AND or OR of LIKE statements) by carefully tuning the code for various cases i.e. (^ exactly matching or substring matching). Similarly, part of regex after the last troubling operators can be handed combination of like statements. Now after splitting the input regex to at most 2 combinations of LIKE statements and regex statement we can use this equivalent query to search for patterns in a column. Example transformation by our algorithm-

Input query-

SELECT string_column FROM TABLE WHERE REGEXP_LIKE (string_column, `^a[pq]c(ab)*(gh)* xyzp*lmn$`)

Transformed Query-

SELECT string_column FROM TABLE WHERE (string_column LIKE `apc` OR string_column LIKE `aqc`) AND (string_column LIKE `lmn`) AND (REGEXP_LIKE (string_column, `^a[pq]c(ab)*(gh)* xyzp*lmn$`))

Here first troubling operator is (ab)* and last troubling operator is p* so entire pattern between these two ((ab)*(gh)* xyzp*) can only be handled by regex. So for pattern before that and after that our algorithm create LIKE statements that can match the pattern.

Guidelines for transforming complex regex query to simpler LIKE query
1. Preprocess the query for handling | operator and break it into multiple simpler regex statements joined by OR operator. 2. Traverse the query to find the first troubling operator and last troubling operator which cannot be converted into LIKE. 3. Simplify the part before the first troubling operator take all possible combinations of patterns and use LIKE statements to match any of those using OR operator. 4. Repeat the same step 3 for the part after the last troubling operator, take all possible combinations of patterns and use LIKE statements to match any of those using OR operator. 5. Join these two set of LIKE statements using AND operator and AND it with regex pattern.

Apart from these troubling operators, there are few operators in regex which cannot be directly converted to a LIKE statement but can be simplified and represented using multiple LIKEs.

1. **Character class or Bracket list**, [...] - Accept ANY ONE of the character within the square bracket, e.g., [aeiou] matches "a", "e", "i", "o" or "u". We can OR multiple LIKE statements to match any one of them. I.e. string_column LIKE `a` OR string_column LIKE `e` string_column OR LIKE `i` OR string_column LIKE `o` OR string_column LIKE `u`.
2. **OR Operator** (|): E.g., the regex four|4 accepts strings "four" or "4". This can also be handled by connection multiple LIKE statements using OR operator.

3. Experiments

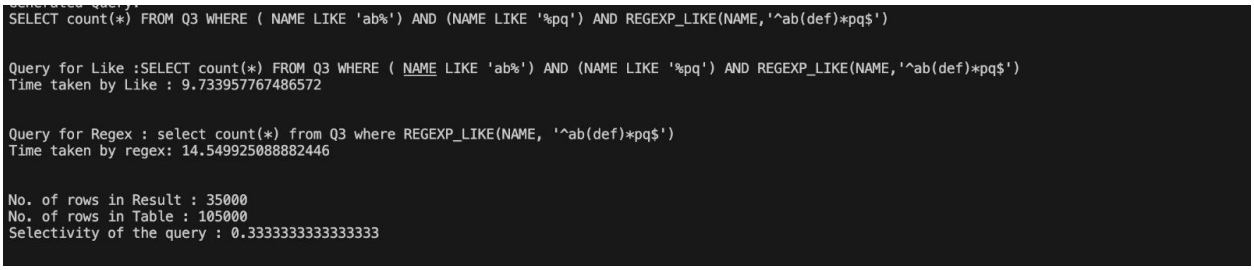
We are using Oracle database for testing as it has support for both LIKE operator as well as REGEXP_LIKE operator. To test the reduction of execution time for pattern matching queries we have performed two types of experiments.

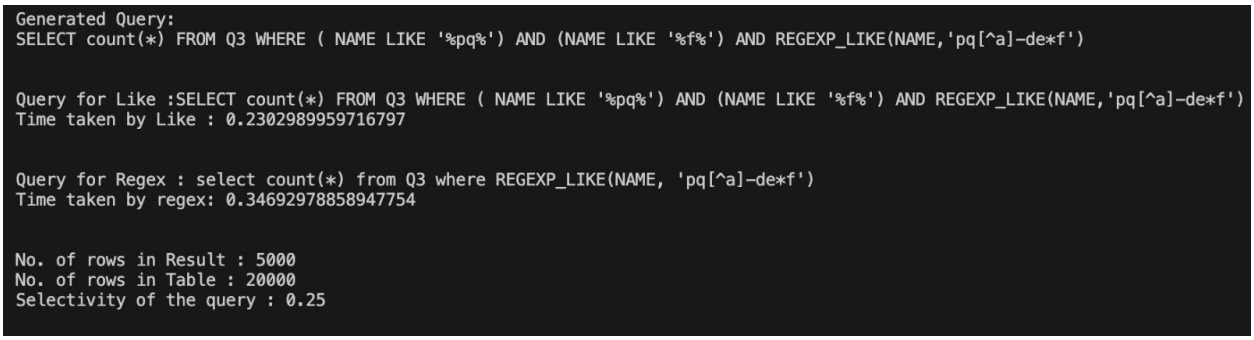
3.1 Experiment 1

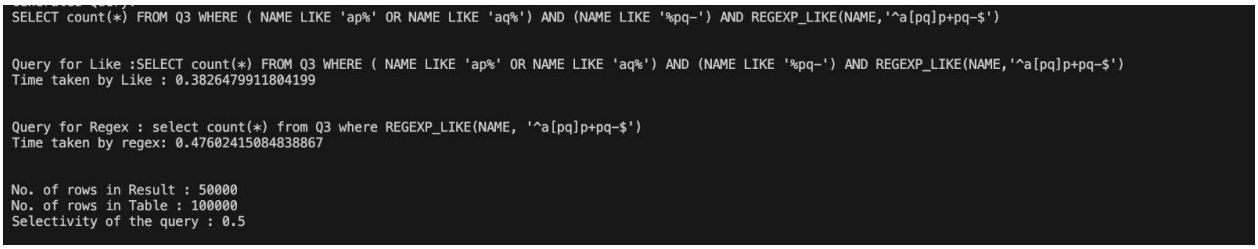
In first experiment we tested multiple different regex queries on a database table of size 100,000 with varying output selectivity (number of rows returned by applying the regex pattern on a column/ total number of rows). These regex queries vary in operators used, size of the regex, complexity of the regex. Operators used in these queries - (),[],*,+,^,$,|,{}.

Each screenshot represents a different query.

1. Query for LIKE- is our transformed query and time taken to run the query is below it.
2. Query for Regex- is the input query and time taken to run the query is below it.







We can observe that for the transformed query can beat the give complex regex query for less selectivity, and for high selectivity it gives nearly similar results.

3.1 Experiment 2

In the second experiment we fixed a query and varied the selectivity to analyze how the selectivity of the query affects the running time of transformed query. Assuming that the query does not use indexes to filter rows before applying REGEXP_LIKE, a full table scan is performed. The REGEXP_LIKE function operates row-by-row. For each row, it evaluates the regex pattern against the column value using a regex engine. This means that every row in the table must be processed regardless of whether it matches the condition. But for low selectivity queries our transformed query will perform better than the complex input query due to the short-circuiting nature of our transformed query.

```
Generated Query:
SELECT count(*) FROM Q3 WHERE ( NAME LIKE 'ab%') AND (NAME LIKE '%-af%') AND REGEXP_LIKE(NAME,'^ab[de]*-af')


Query for Like :SELECT count(*) FROM Q3 WHERE ( NAME LIKE 'ab%') AND (NAME LIKE '%-af%') AND REGEXP_LIKE(NAME,'^ab[de]*-af')
Time taken by Like : 1.3564879894256592


Query for Regex : select count(*) from Q3 where REGEXP_LIKE(NAME, '^ab[de]*-af')
Time taken by regex: 1.2834937572479248


No. of rows in Result : 90000
No. of rows in Table : 100000
Selectivity of the query : 0.9
```

```
Generated Query:
SELECT count(*) FROM Q3 WHERE ( NAME LIKE 'ab%') AND (NAME LIKE '%-af%') AND REGEXP_LIKE(NAME,'^ab[de]*-af')


Query for Like :SELECT count(*) FROM Q3 WHERE ( NAME LIKE 'ab%') AND (NAME LIKE '%-af%') AND REGEXP_LIKE(NAME,'^ab[de]*-af')
Time taken by Like : 0.7411751747131348


Query for Regex : select count(*) from Q3 where REGEXP_LIKE(NAME, '^ab[de]*-af')
Time taken by regex: 0.8281600475311279


No. of rows in Result : 40000
No. of rows in Table : 100000
Selectivity of the query : 0.4
```

```
Generated Query:
SELECT count(*) FROM Q3 WHERE ( NAME LIKE 'ab%') AND (NAME LIKE '%-af%') AND REGEXP_LIKE(NAME,'^ab[de]*-af')


Query for Like :SELECT count(*) FROM Q3 WHERE ( NAME LIKE 'ab%') AND (NAME LIKE '%-af%') AND REGEXP_LIKE(NAME,'^ab[de]*-af')
Time taken by Like : 0.08008098602294922


Query for Regex : select count(*) from Q3 where REGEXP_LIKE(NAME, '^ab[de]*-af')
Time taken by regex: 0.3760099411010742


No. of rows in Result : 10000
No. of rows in Table : 100000
Selectivity of the query : 0.1
```

5. Future work

a. Our algorithm is a greedy algorithm there could be multiple ways to decide whether to convert any part of regex into LIKE or not. One could use a cost function with a crude cost model to calculate the cost incurred in processing regex or part of regex as LIKE statement and based on that we can prune our computation to save time.

b. This work can be extended to include advance operators in regex such as Curb Greediness for Repetition Operators, *?, +?, ??, {m,n}?, {m,}? Greedy quantifiers match as much as possible before backtracking same as working of LIKE % operator while these Lazy operators supported in Oracle DB matches as less as possible in a string. Parenthesized Back References: Use parentheses () to create a back reference. Use $1, $2, ... (Java, Perl, JavaScript) or \1, \2, ... (Python) to retrieve the back references in sequential order.

c. We have not handled character classes for range expression such as [A-Z] and meta characters such as `\d`, `\w`, etc. But these can be handled either in preprocessing step where we can update the regex for range expression to include all the characters in that range i.e. [A-D] can be updated to [ABCD] which our algorithm can handle. Other method is that while traversing the string we convert it into OR of multiple LIKE statements. SQL server LIKE operator provide support even for range expression character classes i.e. [A-D] can be handled by LIKE directly but since SQL server doesn't have any REGEX function, we decided to work with Oracle DB.